EX288

Guide V4 — Labs chronométrés, dépannage rapide et examens blancs

Préparation orientée terrain pour Red Hat Certified OpenShift Application Developer

Positionnement de cette V4

Cette version est conçue comme un pack d'entraînement. Elle réduit la théorie au minimum et concentre l'effort sur des gestes attendus à l'examen : déployer, corriger, valider, relancer, puis revalider.

Point	Valeur
Examen	EX288 — Red Hat Certified OpenShift Application Developer
Version indiquée par Red Hat	OpenShift Container Platform 4.18
Format	Épreuve pratique unique de 3 heures
Préparation recommandée	DO188 et DO288/DO289 selon l'offre suivie
Public visé	Développeurs, DevOps, SRE travaillant sur OpenShift

Références de base utilisées pour cette V4 : page officielle EX288 et offre DO289/DO288, consultées le 7 avril 2026.

1. Comment utiliser cette V4

Ce document est prévu pour une préparation courte et intensive. Le principe est simple :

- choisir un lab chronométré ;
- exécuter les tâches sans aide pendant le temps imparti ;
- valider systématiquement avec oc get, oc describe, logs, events, route et état des pods ;
- noter les erreurs rencontrées et la commande qui a débloqué la situation ;
- rejouer le même lab 24 ou 48 heures plus tard jusqu'à exécution fluide.

Règle d'or

À l'examen, le résultat observable prime. Après chaque action importante, vérifie tout de suite l'effet obtenu. Une modification non testée est une dette de temps.

2. Matrice de couverture officielle → entraînement V4

Objectif officiel Red Hat	Labs V4	Niveau	Signal de validation
Projets, applications mono/multi-conteneurs, probes, console	Lab 1, 2, 3	Critique	Pods Ready, route OK, probes vertes
Helm et Kustomize	Lab 9	Élevé	Rendu final conforme, diff maîtrisé
Images, builds, BuildConfig, registry interne	Lab 4, 5	Critique	Image disponible, déploiement tirant la bonne image
Custom builder workflows, S2I, customisation builder	Lab 6	Critique	Build S2I termine, image générée correctement
Dépannage builds et déploiements	Lab 12 + section pannes	Critique	Cause identifiée, correction confirmée
ImageStreams, triggers	Lab 4, 7	Critique	Nouvelle image détectée, rollout déclenché
ConfigMaps et Secrets	Lab 3	Critique	Variables / fichiers injectés proprement
Hooks et builds déclenchés	Lab 7	Élevé	Hook exécuté, statut contrôlé
Templates OpenShift	Lab 8	Élevé	Paramètres injectés, ressources créées
Pipelines / workflows CI-CD	Lab 10	Élevé	PipelineRun OK, logs compréhensibles
Applications via Operators	Lab 11	Moyen	CR appliqué, ressource

			gérée

3. Labs chronométrés

Les douze labs ci-dessous sont ordonnés pour construire la vitesse d'exécution. Répéter un même lab jusqu'à pouvoir l'achever sans hésitation vaut souvent plus qu'ajouter un nouveau sujet.

Lab	Temps cible	Domaine principal	Niveau
Lab 1	20 min	Projet	Critique
Lab 2	25 min	Multi-conteneurs dans un même Pod	Élevé
Lab 3	20 min	ConfigMaps	Critique
Lab 4	25 min	ImageStreams et registre interne	Critique
Lab 5	25 min	BuildConfig depuis Containerfile/Dockerfile	Critique
Lab 6	30 min	S2I et customisation d'un builder	Critique
Lab 7	20 min	Hooks	Élevé
Lab 8	25 min	Templates OpenShift paramétrés	Élevé
Lab 9	30 min	Helm puis Kustomize	Élevé
Lab 10	30 min	Pipeline Tekton simple à exécuter et dépanner	Critique
Lab 11	15 min	Application déployée via Operator	Critique
Lab 12	40 min	Dépannage transversal	Critique

Lab 1 — Projet, application simple, service, route, probes

Temps cible	20 min
Objectif	Déployer rapidement une application mono-conteneur proprement exposée.
Point de départ	Un projet vide. Tu disposes d'une image HTTP de démo déjà accessible depuis le registre.
Attention	oubli de namespace ; port de container incohérent avec le Service ; route créée avant un service correct ; probes trop agressives.

Tâches à exécuter

1. Créer le projet ex288-lab1.
2. Déployer l'application dans le namespace visé.
3. Créer un Service puis une Route.
4. Ajouter readiness et liveness probes cohérentes.
5. Vérifier l'URL et corriger jusqu'à obtenir une réponse HTTP stable.

Commandes / vérifications réflexes

```
oc project ex288-lab1
oc get all -n ex288-lab1
oc describe deploy/<nom>
oc get route
oc logs deploy/<nom> --tail=50
```

Lab 2 — Multi-conteneurs dans un même Pod

Temps cible	25 min
Objectif	Monter un déploiement avec conteneur principal et sidecar utilitaire.
Point de départ	Image applicative + image sidecar disponibles. Le sidecar sert de collecteur ou de proxy local.
Attention	sidecar qui crash ; mauvaise commande d'entrée ; volume monté sur un seul conteneur ; confusion entre port exposé et targetPort.

Tâches à exécuter

6. Créer le déploiement avec deux conteneurs dans le même Pod.
7. Partager un volume emptyDir entre eux.
8. Vérifier que le conteneur secondaire n'empêche pas le Pod de devenir Ready.
9. Exposer le bon port via Service et Route.

Commandes / vérifications réflexes

```
oc get pods -o wide
oc describe pod/<pod>
oc logs <pod> -c <main>
oc logs <pod> -c <sidecar>
oc exec <pod> -c <main> -- ls -l /tmp
```

Lab 3 — ConfigMaps, Secrets, variables et montages

Temps cible	20 min
Objectif	Injecter proprement de la configuration et des secrets dans une application.

Point de départ	L'application attend des variables d'environnement et un fichier de configuration.
Attention	clé mal orthographiée ; secret monté au mauvais chemin ; application qui lit au démarrage seulement et nécessite un rollout.

Tâches à exécuter

10. Créer une ConfigMap à partir de valeurs simples ou d'un fichier.
11. Créer un Secret opaque.
12. Injecter une partie en env et une partie en volume monté.
13. Redéployer et vérifier que l'application consomme bien les valeurs.

Commandes / vérifications réflexes

```
oc get cm,secret
oc describe deploy/<nom>
oc exec <pod> -- printenv | sort
oc exec <pod> -- cat /etc/config/app.properties
```

Lab 4 — ImageStreams et registre interne

Temps cible	25 min
Objectif	Créer un flux image maîtrisé avec tag et déclenchement propre.
Point de départ	Une image source existe. Il faut la référencer dans OpenShift puis la consommer.
Attention	tag latest confus ; image importée mais déploiement toujours pointé sur l'ancienne référence ; rollout non déclenché.

Tâches à exécuter

14. Créer un ImageStream personnalisé.
15. Importer ou taguer une image dans l'ImageStream.
16. Déployer l'application depuis cet ImageStream.
17. Changer le tag et confirmer le comportement attendu.

Commandes / vérifications réflexes

```
oc get is,istag
oc describe is/<nom>
oc set image ...
oc rollout status deploy/<nom>
```

Lab 5 — BuildConfig depuis Containerfile/Dockerfile

Temps cible	25 min
Objectif	Construire une image avec BuildConfig puis l'utiliser.
Point de départ	Tu disposes d'un dépôt Git avec un Containerfile ou Dockerfile minimal.
Attention	mauvaise branche Git ; fichier de build introuvable ; push vers le mauvais registre ; build complet mais déploiement toujours ancien.

Tâches à exécuter

18. Créer la ressource BuildConfig adaptée.
19. Lancer le build et suivre les logs.
20. Vérifier la publication dans le registre interne.
21. Déployer l'image construite.

Commandes / vérifications réflexes

```
oc start-build <buildconfig> --follow
oc get builds
oc logs build/<nom-build>
oc get is
```

Lab 6 — S2I et customisation d'un builder

Temps cible	30 min
Objectif	Maîtriser le flux S2I et une adaptation simple d'image builder existante.
Point de départ	Un dépôt de code est fourni. Une image builder S2I doit être utilisée puis ajustée.
Attention	builder inadapté ; scripts assemble/run mal copiés ; application qui build mais ne démarre pas.

Tâches à exécuter

22. Créer le build S2I.
23. Personnaliser le comportement assemble/run si le lab le demande.
24. Relancer le build après correction.
25. Déployer et contrôler les logs applicatifs.

Commandes / vérifications réflexes

```
oc new-build --strategy=source ...
oc start-build <bc> --follow
oc logs build/<build>
```

```
oc get buildconfig,builds,imagestreams
```

Lab 7 — Hooks, triggers et relance contrôlée

Temps cible	20 min
Objectif	Déclencher et vérifier automatiquement des builds ou rollouts.
Point de départ	Une application existe déjà. Il faut ajouter les mécanismes de déclenchement utiles.
Attention	trigger présent mais jamais activé ; hook mal référencé ; confusion entre build trigger et rollout trigger.

Tâches à exécuter

26. Ajouter un hook de build ou de déploiement selon l'énoncé.
27. Configurer un trigger image ou config change.
28. Provoquer le changement attendu.
29. Vérifier que le déclenchement réel correspond à l'intention.

Commandes / vérifications réflexes

```
oc describe bc/<nom>
oc describe deploy/<nom>
oc rollout history deploy/<nom>
oc get events --sort-by=.metadata.creationTimestamp | tail
```

Lab 8 — Templates OpenShift paramétrés

Temps cible	25 min
Objectif	Produire un template réutilisable puis l'instancier proprement.
Point de départ	Le lab part d'une application déjà connue à transformer en template.
Attention	paramètre non utilisé ; valeur codée en dur ; oubli d'un objet associé ; template valide mais inutilisable.

Tâches à exécuter

30. Créer le template avec paramètres nom, image, port, mémoire, hostname.
31. Inclure les objets indispensables.
32. Traiter puis appliquer une instance.
33. Vérifier que les paramètres sont bien passés.

Commandes / vérifications réflexes

```
oc process -f template.yaml -p APP_NAME=demo
oc apply -f -
```

```
oc get all
oc get route -o yaml
```

Lab 9 — Helm puis Kustomize

Temps cible	30 min
Objectif	Être à l'aise sur un flux chart + overlay.
Point de départ	Un chart existe déjà mais nécessite adaptation par valeurs et patchs ciblés.
Attention	mauvais namespace cible ; patch Kustomize non appliqué ; conflit entre Helm values et overlay.

Tâches à exécuter

34. Installer ou rendre le chart avec le bon fichier values.
35. Modifier l'exposition, l'image ou les ressources via values.
36. Appliquer ensuite une surcouche Kustomize.
37. Vérifier l'état final obtenu.

Commandes / vérifications réflexes

```
helm template ...
helm install/upgrade ...
kubectl kustomize ... ou oc kustomize ...
oc get all
```

Lab 10 — Pipeline Tekton simple à exécuter et dépanner

Temps cible	30 min
Objectif	Lire, lancer, suivre et corriger un workflow CI/CD simple.
Point de départ	Les Task/Pipeline existent déjà partiellement.
Attention	workspace absent ; image de task erronée ; secret git/registry non monté ; pipeline relancé sans corriger la cause.

Tâches à exécuter

38. Créer les ressources manquantes : PipelineRun, Workspace, Secret ou ServiceAccount si nécessaire.
39. Démarrer le pipeline.
40. Suivre les logs et identifier l'étape en échec.
41. Corriger l'entrée ou la référence fautive puis relancer.

Commandes / vérifications réflexes

```
oc get pipelineruns,taskruns
```

```
tkn pipelinerun logs -f <nom>
oc describe pipelinerun <nom>
oc get events
```

Lab 11 — Application déployée via Operator

Temps cible	15 min
Objectif	Savoir consommer un Operator et créer la ressource attendue.
Point de départ	L'Operator est disponible ou déjà installé selon le lab.
Attention	mauvaise CR apiVersion ; namespace d'installation confus ; attente trop courte avant validation.

Tâches à exécuter

42. Vérifier l'Operator pertinent.
43. Appliquer une Custom Resource simple.
44. Contrôler les ressources créées et la santé de l'application.

Commandes / vérifications réflexes

```
oc get csv,subscriptions
oc get crd | grep -i ...
oc get <cr-kind>
oc describe <cr-kind>/<nom>
```

Lab 12 — Dépannage transversal

Temps cible	40 min
Objectif	Traiter un lot de pannes mêlant build, image, service et configuration.
Point de départ	Plusieurs objets existent déjà mais le rendu final n'est pas conforme.
Attention	corriger trop tôt les symptômes visibles sans remonter à la première cause ; négliger les Events ; oublier le registre interne.

Tâches à exécuter

45. Identifier les symptômes prioritaires.
46. Réparer d'abord ce qui bloque la création d'image ou le démarrage des pods.
47. Réparer ensuite l'exposition réseau.
48. Faire une validation complète de bout en bout.

Commandes / vérifications réflexes

```
oc get all,cm,secret,is,builds
```

oc describe ...

oc logs ...

oc get events --sort-by=.lastTimestamp

4. Dépannage rapide — 15 pannes à savoir traiter

Symptôme	Cause probable	Réflexe de diagnostic
Pod en CrashLoopBackOff	Commande d'entrée, variable absente, port attendu différent	oc describe pod ; oc logs ; vérifier env et mount
Pod Running mais non Ready	readiness probe incorrecte	oc describe pod ; comparer path/port avec l'application
Route répond en erreur	service/targetPort incohérent	oc get svc,route,endpoints ; aligner targetPort
Build échoue dès le clone Git	URL, branche ou secret git erroné	oc logs build/<nom> ; corriger source/credentials
Build OK mais ancienne version déployée	déploiement toujours lié à une ancienne image	contrôler ImageStreamTag et référence image du déploiement
Image importée mais pas de rollout	trigger absent ou non activé	oc describe deploy ; ajouter le bon trigger
Secret créé mais application ne voit rien	mauvais nom de clé ou mauvais mountPath	oc describe deploy ; oc exec dans le pod
S2I produit une image qui ne démarre pas	scripts assemble/run ou builder inadaptés	relire logs build puis logs runtime
Pipeline Tekton échoue avant la première task	ServiceAccount, Workspace ou permission manquante	oc describe pipelinerun ; tkn logs
Template processé mais invalide	paramètres non déclarés ou objets manquants	oc process -f template.yaml ; corriger placeholders
Operator installé mais CR ignorée	apiVersion/kind erroné ou namespace incorrect	oc get crd ; vérifier la CR officielle
Registry interne inaccessible	authentification ou configuration du registry	vérifier login, secret, référence registry
Multi-conteneur : un sidecar bloque tout	healthcheck ou commande du sidecar défaillante	isoler les logs par conteneur
Kustomize n'applique pas le patch	cible mal identifiée	vérifier name/namespace/kind du patch
Hook configuré mais jamais vu en exécution	mauvais type de hook ou mauvais moment de déclenchement	oc describe build/deploy ; générer explicitement l'événement déclencheur

Séquence de diagnostic recommandée

1) oc get all ; 2) oc describe de la ressource fautive ; 3) logs ; 4) events triés par date ; 5) correction minimale ; 6) revalidation immédiate.

5. Examen blanc A — 3 heures

Pondération ci-dessous : strictement indicative pour ton entraînement. Red Hat ne publie pas la pondération détaillée des items ; le résultat officiel est communiqué comme score total.

Tâche	Poids entraînement	Validation attendue
Créer un projet, déployer une application simple, configurer Service, Route et probes.	18	Résultat observable conforme + contrôle par oc
Créer ConfigMap et Secret, puis les injecter proprement.	14	Résultat observable conforme + contrôle par oc
Créer un ImageStream personnalisé et déployer depuis ce flux.	14	Résultat observable conforme + contrôle par oc
Construire une image depuis un dépôt Git avec BuildConfig.	16	Résultat observable conforme + contrôle par oc
Configurer un build S2I et corriger un échec de build fourni.	18	Résultat observable conforme + contrôle par oc
Mettre en place un trigger d'image puis démontrer le rollout.	10	Résultat observable conforme + contrôle par oc
Créer un template paramétré pour l'application.	14	Résultat observable conforme + contrôle par oc
Déployer la même application via Helm puis ajuster une valeur demandée.	12	Résultat observable conforme + contrôle par oc
Appliquer un patch Kustomize sur le déploiement.	10	Résultat observable conforme + contrôle par oc
Exécuter un PipelineRun Tekton fourni et corriger l'étape qui échoue.	16	Résultat observable conforme + contrôle par oc
Traiter deux pannes transversales fournies par l'énoncé.	18	Résultat observable conforme + contrôle par oc

Règle d'exécution : lire tout le sujet pendant 10 à 15 minutes, marquer les tâches rapides, sécuriser ensuite les points les plus accessibles avant d'attaquer les scénarios de correction plus risqués.

6. Examen blanc B — 3 heures

49. Sujet 1. Application multi-conteneurs avec volume partagé et exposition réseau correcte.

50. Sujet 2. Image custom bâtie à partir d'un Containerfile et publiée dans le registre interne.
51. Sujet 3. Customisation d'un builder S2I existant puis rebuild.
52. Sujet 4. Hook ou trigger à ajouter puis à vérifier.
53. Sujet 5. Template OpenShift multi-objets.
54. Sujet 6. Pipeline Tekton partiellement fourni à terminer.
55. Sujet 7. Déploiement d'une application gérée par Operator.
56. Sujet 8. Lot final de dépannage.

Objectif du blanc B : varier les combinaisons sans changer les gestes fondamentaux.

7. Plan d'attaque sur 7 jours

Jour	Bloc	Labs	Cible
J1	Socle	1 + 3	Déployer vite et sans erreur de namespace
J2	Images	4 + 5	Maîtriser ImageStreams, builds et registry
J3	S2I / triggers	6 + 7	Corriger un build puis prouver le déclenchement
J4	Packaging	8 + 9	Template, Helm, Kustomize sans hésitation
J5	Pipelines / Operator	10 + 11	Lire vite et corriger le minimum nécessaire
J6	Dépannage	12 + section pannes	Réduire le temps de diagnostic
J7	Simulation	Blanc A ou B	Tenir 3 heures avec méthode

8. Commandes réflexes

```
oc project <ns>
oc get all -n <ns>
oc get route,svc,endpoints
oc describe pod/<pod> ; oc describe deploy/<deploy>
oc logs <pod> ; oc logs build/<build> ; tkn pipelinerun logs -f <name>
oc get events --sort-by=.metadata.creationTimestamp | tail -n 30
oc rollout status deploy/<name> ; oc rollout history deploy/<name>
oc exec <pod> -- env | sort ; oc exec <pod> -- sh
oc process -f template.yaml -p KEY=VALUE
helm template ... ; helm upgrade --install ...
```

9. Références officielles à relire avant inscription

- EX288 — Red Hat Certified OpenShift Application Developer exam
- DO289 / DO288 — OpenShift Developer II: Building and Deploying Cloud-native Applications
- DO188 — OpenShift Development I: Introduction to Containers with Podman

Toujours recontrôler la page EX288 juste avant l'inscription : Red Hat indique explicitement que les objectifs peuvent évoluer et que ces changements sont publiés à l'avance.